Ex. No. 8	Data Persistence using SQLite and Firebase
Date of Exercise	12/10/26
GitHub Host Link	https://github.com/jomiwilson/flutter_lab_exp8

Aim

To implement data persistence using a local SQLite database and cloud Firebase database.

Description

This experiment introduces data persistence in Flutter using local and cloud storage. Student records are stored locally in an SQLite database using the sqflite package with CRUD operations. Student data is also stored in and retrieved from a Firebase cloud collection.

Tools/Software Required: Flutter

Procedure:

This experiment is larger than the previous experiments, so perform it in two parts.

PART A – SQLite

Step 1

Create:

```
flutter create sqlite_app
cd sqlite_app
```

Step 2 – Add packages

Open pubspec.yaml.

Add suitable SQLite packages such as:

```
dependencies:
  sqflite:
  path:
```

Then:

```
flutter pub get
```

Step 3 – Create database helper

Create:

```
lib/database_helper.dart
```

Step 4 – Create database

Create a database containing:

```
students
id
```

```
name
course
```

Step 5 – Implement CRUD

Implement:

```
CREATE
READ
UPDATE
DELETE
```

Step 6 – Insert student

Collect information from TextFields and insert it into SQLite.

Step 7 – Display students

Read records from SQLite and display them using:

```
ListView.builder()
```

Step 8 – Test persistence

Close the application.

Run again:

```
flutter run
```

Verify that previously stored data is still available.

PART B – Firebase

Step 9 – Create Firebase project

Go to the Firebase Console and create a project.

Step 10 – Connect Flutter application

Configure Firebase for the Flutter application.

Step 11 – Add Firebase packages

Add the required Firebase packages to:

```
pubspec.yaml
```

Step 12 – Initialize Firebase

Initialize Firebase before running the application.

Step 13 – Create cloud collection

Create a collection such as:

```
students
with fields:
name
course
```

Step 14 – Insert data

Send student information to Firebase.

Step 15 – Retrieve data

Read Firebase records and display them in Flutter.

Program

pubspec.yaml (dependencies)

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.3
  sqflite_common_ffi_web: ^0.4.5
  path: ^1.9.0
  firebase_core: ^3.6.0
  cloud_firestore: ^5.4.4
```

lib/database_helper.dart

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('students.db');
    return _database!;
  }

  Future<Database> _initDB(String fileName) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, fileName);
    return await openDatabase(path, version: 1, onCreate: _createDB);
  }

  // Create the students table
  Future<void> _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        course TEXT NOT NULL
      )
    ''');
  }
```

```
  // CREATE
  Future<int> insertStudent(String name, String course) async {
    final db = await database;
    return await db.insert('students', {'name': name, 'course': course});
  }

  // READ
  Future<List<Map<String, dynamic>>> getStudents() async {
    final db = await database;
    return await db.query('students', orderBy: 'id');
  }

  // UPDATE
  Future<int> updateStudent(int id, String name, String course) async {
    final db = await database;
    return await db.update(
      'students',
      {'name': name, 'course': course},
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  // DELETE
  Future<int> deleteStudent(int id) async {
    final db = await database;
    return await db.delete('students', where: 'id = ?', whereArgs: [id]);
  }
}
```

lib/main.dart

```
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';
import 'database_helper.dart';
import 'firebase_options.dart';
import 'firebase_page.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // SQLite on the web uses the sqflite web (WASM) implementation
  if (kIsWeb) {
    databaseFactory = databaseFactoryFfiWebNoWebWorker;
  }
  // Initialize Firebase before running the application
  try {
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    ).timeout(const Duration(seconds: 5));
  } catch (e) {
```

```
    debugPrint('Firebase not configured yet: $e');
  }
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: SQLitePage(),
    );
  }
}

// PART A - SQLite
class SQLitePage extends StatefulWidget {
  const SQLitePage({super.key});

  @override
  State<SQLitePage> createState() => _SQLitePageState();
}

class _SQLitePageState extends State<SQLitePage> {
  final nameController = TextEditingController();
  final courseController = TextEditingController();
  List<Map<String, dynamic>> students = [];
  int? editingId;

  @override
  void initState() {
    super.initState();
    loadStudents();
  }

  Future<void> loadStudents() async {
    final data = await DatabaseHelper.instance.getStudents();
    setState(() {
      students = data;
    });
  }

  Future<void> saveStudent() async {
    if (nameController.text.isEmpty || courseController.text.isEmpty) return;
    try {
      if (editingId == null) {
        await DatabaseHelper.instance
            .insertStudent(nameController.text, courseController.text);
      } else {
        await DatabaseHelper.instance.updateStudent(
            editingId!, nameController.text, courseController.text);
```

```
        editingId = null;
      }
      nameController.clear();
      courseController.clear();
      await loadStudents();
    } catch (e) {
      debugPrint('SQLite error: $e');
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Could not save: $e')),
      );
    }
  }

  void editStudent(Map<String, dynamic> student) {
    setState(() {
      editingId = student['id'];
      nameController.text = student['name'];
      courseController.text = student['course'];
    });
  }

  Future<void> deleteStudent(int id) async {
    await DatabaseHelper.instance.deleteStudent(id);
    loadStudents();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Exp 8: Data Persistence (SQLite)'),
        actions: [
          IconButton(
            icon: const Icon(Icons.cloud),
            tooltip: 'Firebase',
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const
FirebasePage()),
              );
            },
          ),
        ],
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: nameController,
              decoration: const InputDecoration(labelText: 'Student Name'),
```

```
            ),
            TextField(
              controller: courseController,
              decoration: const InputDecoration(labelText: 'Course'),
            ),
            const SizedBox(height: 12),
            SizedBox(
              width: double.infinity,
              child: ElevatedButton(
                onPressed: saveStudent,
                child: Text(editingId == null ? 'Add Student' : 'Update
Student'),
              ),
            ),
            const SizedBox(height: 12),
            const Align(
              alignment: Alignment.centerLeft,
              child: Text(
                'Saved SQLite Records',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ),
            Expanded(
              child: ListView.builder(
                itemCount: students.length,
                itemBuilder: (context, index) {
                  final s = students[index];
                  return ListTile(
                    leading: CircleAvatar(child: Text('${s['id']}')),
                    title: Text(s['name']),
                    subtitle: Text(s['course']),
                    onTap: () => editStudent(s),
                    trailing: IconButton(
                      icon: const Icon(Icons.delete, color: Colors.red),
                      onPressed: () => deleteStudent(s['id']),
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/firebase_page.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';

// PART B - Firebase (Cloud Firestore)
```

```
class FirebasePage extends StatefulWidget {
  const FirebasePage({super.key});

  @override
  State<FirebasePage> createState() => _FirebasePageState();
}

class _FirebasePageState extends State<FirebasePage> {
  final nameController = TextEditingController();
  final courseController = TextEditingController();
  final CollectionReference studentsRef =
      FirebaseFirestore.instance.collection('students');

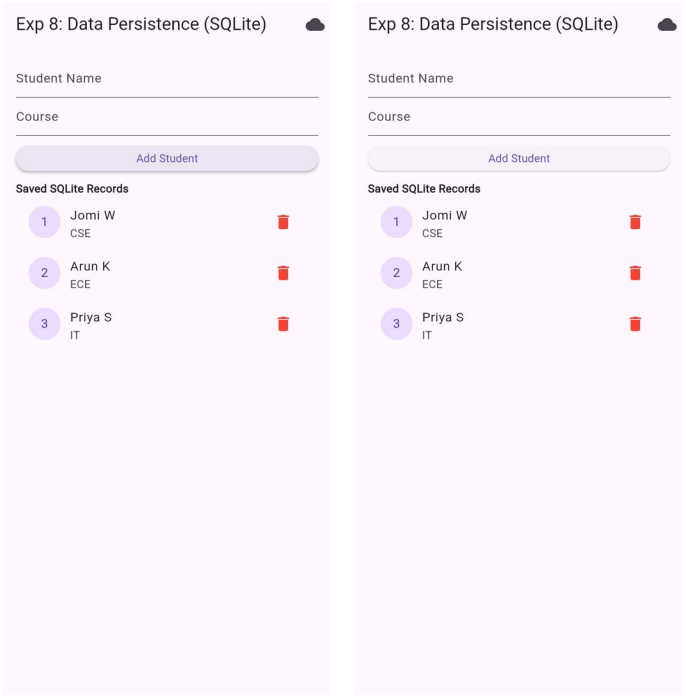
  // Insert data into the Firebase collection
  Future<void> addStudent() async {
    if (nameController.text.isEmpty || courseController.text.isEmpty) return;
    await studentsRef.add({
      'name': nameController.text,
      'course': courseController.text,
    });
    nameController.clear();
    courseController.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Exp 8: Firebase Storage'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: nameController,
              decoration: const InputDecoration(labelText: 'Student Name'),
            ),
            TextField(
              controller: courseController,
              decoration: const InputDecoration(labelText: 'Course'),
            ),
            const SizedBox(height: 12),
            SizedBox(
              width: double.infinity,
              child: ElevatedButton(
                onPressed: addStudent,
                child: const Text('Send to Firebase'),
              ),
            ),
            const SizedBox(height: 12),
            // Retrieve and display Firebase records
            Expanded(
```

```
              child: StreamBuilder<QuerySnapshot>(
                stream: studentsRef.snapshots(),
                builder: (context, snapshot) {
                  if (!snapshot.hasData) {
                    return const Center(child: CircularProgressIndicator());
                  }
                  final docs = snapshot.data!.docs;
                  return ListView.builder(
                    itemCount: docs.length,
                    itemBuilder: (context, index) {
                      final data = docs[index].data() as Map<String,
dynamic>;

                      return ListTile(
                        leading: const Icon(Icons.cloud_done),
                        title: Text(data['name'] ?? ''),
                        subtitle: Text(data['course'] ?? ''),
                      );
                    },
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output



Result

A Flutter application demonstrating data persistence was developed successfully. Student records were stored in a local SQLite database using CREATE, READ, UPDATE and DELETE. The stored data remained available after the application was closed and run again. Student information was sent to a Firebase collection and retrieved and displayed in the application.